## WHAT IS "TOON SHADERS PRO FOR URP"?

*Toon Shaders Pro for URP* is a collection of shaders for creating toon-styled games, in the cel-shaded style made popular by games like *Jet Set Radio* and *The Legend of Zelda: Breath of the Wild*. The main toon shader gives you control over the different kinds of lighting applied to objects, including terrains, and the outline shader gives you several options for rendering lines along the edges of objects.

The latest version of this asset has been tested with [ **Unity 2022.3.0f1, URP 14.0.7** ] and [ **Unity 6000.0.23f1, URP 17.0.3** ].

## SUPPORT

Sometimes things break! If you've encountered an error and this README doesn't have the answers (or if you have feature requests), then pop me an email at **danielilett+toonshaders@gmail.com** and I'll try to sort you out. Please:

- **Clearly** describe the problem you are having and what steps I can take to reproduce the error.
- Include the **Unity version you are using.**
- Also include the **package version** you are using (you can find this in the Package Manager).
- Include the **render path** (i.e. Forward, Forward+, Deferred) – this option is on your URP settings asset.
- A**ttach images or short videos** where necessary to describe your problem.

Following these steps will help me fix your issue as quickly as possible!

# SETUP

**Folder Structure**: Upon installing the pack, all the assets will be contained in the "Toon Shaders Pro" folder. All demo scenes are included under "Toon Shaders Pro/Demo", containing an example scene with materials and post processing volumes configured to work with this asset pack.

Most of the shaders are included inside "Toon Shaders Pro/Shaders". The post processing assets are included in the "Retro Shaders Pro/Shaders/Outline" folder.

**Warning**: There is a chance that you will encounter problems when you import the pack due to the Shader Graph version of the toon shader. This problem relates to the shader keywords used for lighting. If you have this issue, please go into *Preferences -> Shader Graph* (before Unity 6) or *Project Settings -> Shader Graph* (Unity 6 and after) to increase the shader variant limit – this might need to be at least 512.

## TOON SHADER FOR MESHES

There are two versions of the main Toon shader which are intended to be used for regular objects in your game.

For most objects, you should just create a material with the "**Toon Shaders Pro/URP/Toon"** shader.

If you want to create your own shaders which have toon capabilities, you may find it helpful to duplicate the "**Toon Shaders Pro/URP/Toon (Graph)**" Shader Graph file and modify that.

## TOON SHADER FOR TERRAINS

There is also a version of the toon shader specifically for terrains. This version should only be attached to terrains, and it will use the splatmaps applied to the terrain to determine its albedo color. The "**Toon Shaders Pro/URP/Toon (Terrain)**" shader should be used for the material, which will let you modify the lighting, just like regular objects.

## OUTLINE POST PROCESS

This shader pack uses **Universal Render Pipeline**'s **ScriptableRenderFeature** functionality for the custom Outline post processing effect. The Unity documentation will outline the basics of URP if you're not familiar with how to create custom renderers.

Please follow these steps to enable an effect in your scene:

- Find your **URP Renderer Asset** and add the effect(s) you wish to use in the **Renderer Features** section at the bottom.
    - This is most commonly found in the Assets/Settings folder if you created a new project using the URP template from the Unity Hub.
    - This asset will be named something like "UniversalRP-HighQuality" (Unity versions 2022.3 and prior) or "PC_RPAsset" (Unity 6) by default.
    - *Toon Shaders Pro* also includes a ready-made asset named "**URP-Toon-Renderer**" in the Demo folder, which has the Outline effect pre-added.

- Create a volume profile asset via **Create -> Volume Profile** and add the Outline effect (and any other effects you want to use) to the profile.
- Add a volume to your scene via *GameObject -> Volume* and attach the volume profile.
- Tweak the settings on your volume profile as desired.

### RENDER GRAPH SUPPORT (UNITY 6)

Effects which use the Renderer Feature API will require URP **Render Graph** in the future. *Toon Shaders Pro* supports Render Graph and non-Render Graph workflows.

You can disable Render Graph via *Project Settings -> Graphics -> Pipeline Specific Settings -> URP*. You will find a checkbox to disable Render Graph near the bottom of the window. However, this shouldn't be necessary unless your project uses external effects which rely on the non-Render Graph workflow.

### ADDITIONAL WARNINGS

These shaders are designed for **linear color space**, so you may encounter issues in gamma space. To swap between color spaces, go to *Project Settings->Player->Other Settings* and find the **Color Space** dropdown option.

## ASSETS INCLUDED

The following assets are included in the asset pack:

### TOON SHADER

The main Toon shader lets you separate the diffuse, specular, and rim lighting and the shadows shown on the object and apply cutoff thresholds for each. You can also specify tint values for the areas of the mesh in light and darkness, or apply a normal map for even more interesting lighting.

#### SURFACE OPTIONS

- **Surface Type** – Toggle between *Opaque* and *Transparent* rendering.
- **Render Face** – Choose whether to render *Front*, *Back*, or *Both* faces.
- **Alpha Clip** – Toggle whether the shader should apply the *Alpha Clip Threshold*.
- **Alpha Clip Threshold** – Pixels with final base color alpha values below this threshold will be culled if *Alpha Clip* is enabled.
- **Receive Shadows** – Toggle whether realtime shadows should be applied to the object.
- **Shadow Thresholds** – Provide a toon-style cutoff to the realtime shadows.

#### TOON PROPERTIES

##### DIFFUSE SETTINGS

- **Use Second Threshold** – When ticked, the shader introduces a third lighting tint value for diffuse lighting and lets you specify two thresholds: one for dark -> midtones, and another for midtones -> light.
- **Base Color** – The albedo color of the object.
- **Base Texture** – An albedo texture to apply more color detail than *Base Color* alone.

- **Base Texture Tiling and Offset** – Lets you scale and translate the texture across the object surface. Note that these values are also used to scale and translate *Specular Map* and *Normal Map*.
- **Light Tint** – Color multiplier applied to fully-lit areas of the object.
- **Middle Tint** – Only active if *Use Second Threshold* is ticked. Color multiplier applied to midtones on the object.
- **Shadow Tint** – Color multiplier applied to shadowed regions of the object.
- **Ambient Light Strength** – How strongly Unity's default ambient lighting is applied to the object.
- **Diffuse Thresholds** – Provide a toon-style cutoff to the diffuse lighting. Diffuse lighting comes from the angle between the light and the normal vector of the object surface.

The top-right of the mesh uses the Light Tint, with a small rim highlight. The middle section uses the Middle Tint. The lowest section uses the Shadow Tint.

### SPECULAR SETTINGS

- **Specular Map** – Control the specular strength at different parts of the object using a texture.
- **Specular Strength** – Control the global specular strength of the object.
- **Specular Offset Noise Map** – Use a noise texture, such as Perlin noise, to modify the shape of the specular highlights.
- **Specular Offset Noise Strength** – Controls the strength of the offset applied to specular light values.
- **Specular Color** – Multiplier for specular light.
- **Glossiness** – Higher glossiness makes the object look shiny and results in smaller highlights.
- **Specular Thresholds** – Provide a toon-style cutoff to the specular lighting. Specular lighting arises due to the angle between the viewer and the light rays reflected from the light source in the normal vector of the object's surface.

The toon shader supports multiple additional lights and applies all these thresholds for each light.

### RIM LIGHTING SETTINGS

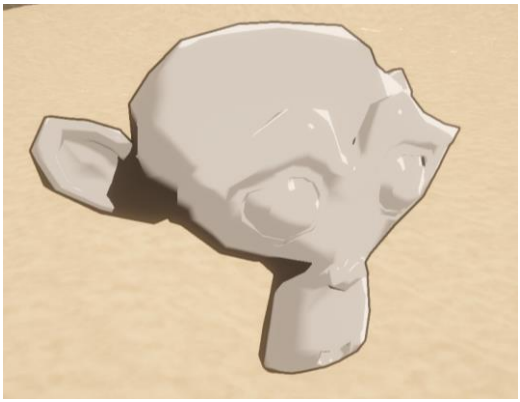
- **Rim Color** – Multiplier for rim light.
- **Rim Thresholds** – Provide a toon-style cutoff to the rim lighting. Rim lighting (also called Fresnel) comes from the angle between the viewing angle and the normal vector of the surface. Typically, looking at the object at shallow angles results in high rim lighting.
- **Rim Extension** – Lets you extend rim lighting across the surface further than usual.

### NORMAL MAPPING SETTINGS

- **Normal Map** – Normal maps modify the surface normals for finer details that are difficult to capture with mesh geometry alone.
- **Normal Strength** – Larger values have a more pronounced effect on the surface normals.

## TIPS FOR USE

- Each of the thresholds use smoothstep under the hood so that you can soften the transition between light and dark. If you want to use a single, hard step transition, use the same value for both min and max.
- Lighting is additive, so the diffuse, specular, and rim contributions are added together. Contributions from each light are added together. Consequently, using many additional lights might end up with exceptionally bright objects.
- You can achieve a clean, block-color aesthetic by leaving the Base Map and Normal Map blank and attaching the Outline effect also included in this pack. However, normal mapping can achieve quite striking visuals when combined with toon lighting.



You can use wider thresholds for a smoother look, slightly different to the traditional hard toon cutoff.

## TOON GRAPH

This is largely a copy of the base Toon shader, but made with Shader Graph. Most of the toon lighting logic is implemented in a subgraph named "**CalculateToonLighting**". I've included this to make it simpler to make your own effects.

All you need to do is duplicate this graph and add your own nodes however you want. Or, you can drag the CalculateToonLighting subgraph into any graph. Be warned, though: it has a lot of inputs! I've tried to include sensible defaults where possible.

## TOON TERRAIN SHADER

The terrain version of the shader has exclusive additional features that only make sense with terrains, and is missing the Base Map option, as the base/albedo texture now comes from the terrain splatmaps and layers.

- **Use Stochastic Texturing** – When enabled, the shader uses UV offsets to prevent the input textures from looking tiled, which is a common problem with terrain textures. This method uses three times as many texture samples for each terrain base layer.

## OUTLINE POST PROCESS

The Outline effect is a collection of multiple outline rendering algorithms which each have different settings and use cases.

The options provided with this feature change quite drastically based on the *Outline Type*, but I will list them all here for convenience.

### RENDERING OPTIONS

- **Render Pass Event** – Choose whether to render the outlines before or after URP's internal post processing effects. That includes effects such as color correction and bloom.
- **Outline Type** – Choose which outline algorithm to use. There are six choices:
    - **No Outlines** – Don't render any outlines.
    - **Depth Normal Outlines** – Detect small gradients in the color, depth, and normal vector information across each pixel to find edges.
    - **High Quality Masked Object Outlines** – Render specific object layers to a separate mask texture and find edges in that mask, allowing for varying thickness and a greater level of customization than other algorithms.
    - **Pixel Width Masked Object Outlines** – Use the same object mask as above, but use a cheaper algorithm for pixel-width outlines.
    - **Hull Outlines** – Render each mesh a second time while inverting and inflating the mesh hull.
    - **Debug Outline Mask** – Render only the object mask used for the mask-based effects to the screen. Helpful for finding objects which may have been incorrectly assigned to the wrong layer.

### OUTLINE OPTIONS

- **Outline Color** – Color applied to the outlines. Some effects may use the transparency value.

A detailed normal map combined with high-sensitivity normal-based outlines can result in a sketch-like effect.

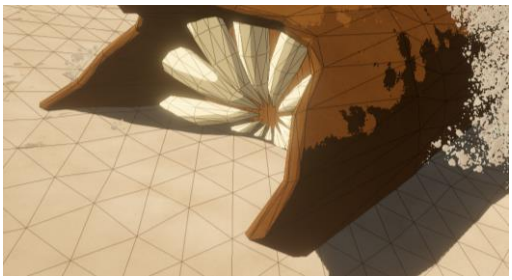
### DEPTH-NORMALS OPTIONS

- **Color Sensitivity** – Color gradients greater than this threshold will be detected as edges.
- **Color Strength** – How strongly a detected color-edge should contribute to the final edge-detection value.
- **Depth Sensitivity** – Depth gradients greater than this threshold will be detected as edges.
- **Depth Strength** – How strongly a detected depth-edge should contribute to the final edge-detection value.
- **Normal Sensitivity** – Normal vector gradients greater than this threshold will be detected as edges.

- **Normal Strength** – How strongly a detected normal-edge should contribute to the final edge-detection value.
- **Depth Threshold** – Pixels with a depth higher than this threshold are never detected as edges.

## OBJECT MASK OPTIONS

- **Object Mask** – Set a layer mask to determine which object layers get rendered into the mask texture.
- **Mask Ignore Depth** – If enabled, objects get rendered into the mask texture without checking for depth. If rendering many objects, this likely means objects will be rendered out of order. Useful if you render only one object and you want it to be seen through walls.
- **Mask Drawing Mode** – Choose whether to render outlines along the outer edge of each mesh, along the edge of each triangle, or around the collective area of objects drawn into the mesh.
- **Light Modes** – Determines which objects get drawn into the mask, based on which shader they are using originally. The tooltip provides useful context for this setting.
- **Render Queue** – Choose whether to render only opaques, only transparents, or all objects into the mask. However, only opaque objects are properly supported.



Triangle-based outlining can be useful for debugging or for a very unconventional stylized effect.

## MASK OUTLINE OPTIONS

- **Masked Outline Thickness** – Set the thickness of the outlines. Larger values get more expensive, so try to keep this as low as possible.
- **Masked Outline Smoothing** – Decreases the outline contribution of further away pixels for a false smoothing effect.
- **Outline Draw Sides** – Choose to draw objects 'inside' the mask, 'outside' it, or on both 'sides'. Drawing on both sides results in a thicker outline.
- **Outline Fade Start** – Start to fade out the outlines when the object is this many Unity units from the camera.
- **Outline Fade End** – When the object is at this distance from the camera, the objects become totally invisible.

Mask-based outlines render over the particles, because they do not render depth and therefore do not influence mask rendering.

## NORMALS OPTIONS

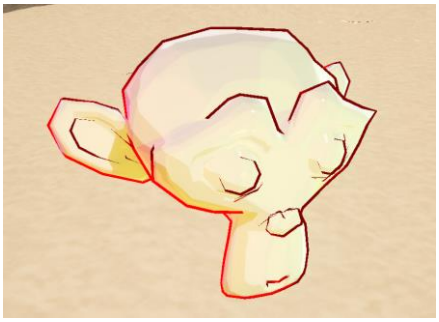
- **Use Depth Normals** – When using mask-based outlines, choose whether to layer normal-based edge detection for details 'inside' the mask.

### HULL OUTLINE OPTIONS

- **Outline Thickness** – How far the outlines extend away from the object.
- **Outline Transparency** – Choose whether to render the outlines with transparency. Since this method uses actual mesh geometry, this has an impact on rendering order.
- **Outline Lighting** – Should the outlines use simple diffuse lighting based only on the main directional light?
- **Flip Outline Direction** – When lighting is being used, should the direction of the normal vector be inverted?
- **Outline Min Lighting** – Set a minimum lighting value to use for the diffuse outline lighting.

### TIPS FOR USE

- Rendering outlines *before* other post processing effects means that they are subject to those effects. For example, color correction and bloom effects will apply. This may result in a soft, stylized look for your outlines. Otherwise, if you want the outlines to use exactly the color you specify, you should render them *after*.
- For the *Light Mode* setting, Unity restricts the choice of objects to only those objects which use a shader with a light mode in this list. Most lit shaders use *UniversalForward*. Most unlit shaders don't use a tag, which defaults to *SRPDefautlUnlit*. The toon shader in this pack uses *UniversalForwardOnly*. Shaders which support deferred rendering use *UniversalGBuffer*.



Red hull outlines with lighting enabled results in a stylish effect.

## SPECIAL THANKS

Many thanks to:

- ambientCG for many of the CC0 licensed textures used in the demo
- OpenGameArt for some CC0 licensed audio used in the demo
- Kenney for some smoke particle textures used in the demo
- Blender Foundation for the Suzanne monkey model used in the demo